

EMAIL AI EXTRACTION AGENT

Complete Technical Guide — LLM Flow · Framework · Code

7

Core Steps

3

Main Layers

1

API Call

∞

Emails

0	
1	Architecture Overview & Core Concept
0	
2	The LLM Flow — How the AI Thinks
0	
3	Framework Flow — How the Code Works
0	
4	Step-by-Step: Building the Agent
0	
5	The System Prompt — Agent's Brain
0	
6	API Call & JSON Parsing
0	
7	UI & Results Rendering
0	
8	Error Handling & Edge Cases
0	
9	Complete Code Reference
1	
0	Extending the Agent

STEP 01 · ARCHITECTURE OVERVIEW & CORE CONCEPT

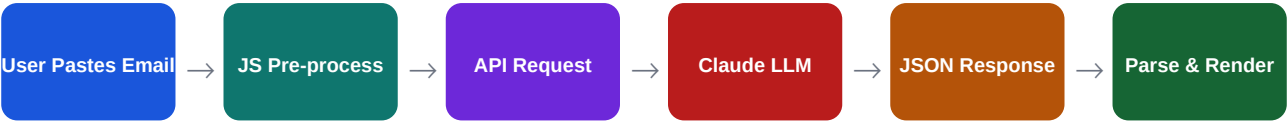
What Is This Agent?

The Email AI Extraction Agent is a single-page intelligent application that accepts raw email text as input, processes it through a carefully engineered LLM prompt, and returns a richly structured JSON object containing every piece of important information — all rendered instantly in a clean UI.

The Three-Layer Architecture

Layer	Name	Responsibility	Technology
1 — Input	Email Intake	Accept raw email text, pre-process it	HTML textarea
2 — Brain	LLM Processing	Understand, classify, extract information	Anthropic Claude
3 — Output	Results Rendering	Parse JSON, display categorised cards	JavaScript DOM

End-to-End Data Flow



Each arrow represents a data handoff. The entire pipeline completes in 2–5 seconds.

Key Design Principle

The agent has NO state between sessions. Every extraction is a fresh, stateless API call. This makes it reliable, fast, and easy to scale — no database, no session management. All intelligence lives inside the LLM prompt (the System Prompt).

STEP 02 · THE LLM FLOW — HOW THE AI THINKS

How Claude Processes the Email

When the email reaches Claude, the LLM does not simply 'search' for keywords. Instead, it performs deep language understanding across several internal stages — all within a single API call.

1. Context Setting

Claude reads the SYSTEM PROMPT first. This tells it: 'You are an expert email analyst. Your job is to extract structured information and return valid JSON only.' This sets the model's entire mindset, role, and output format before it sees any email.

2. Email Ingestion

The raw email text is passed as the USER message. Claude reads the entire email holistically — understanding sender intent, urgency, structure, and relationships between pieces of information. It does not process line-by-line; it understands the whole meaning first.

3. Entity Recognition

Claude identifies named entities: people (sender, recipients, owners), dates and deadlines, URLs and hyperlinks, monetary values, product/project names, and organizations. These map to fields like 'action_items.owner', 'links.url', and 'important_text'.

4. Semantic Classification

Each sentence is classified by importance and type. Is this a heading, a major point, an action item, or just background context? Claude weighs urgency language ('REQUIRED', 'deadline', 'critical') to set the urgency field: low / medium / high.

5. Structured Generation

Claude generates the output as a JSON object, field by field, following the exact schema defined in the system prompt. It only outputs fields with real content — never hallucinating data that isn't present in the email.

6. JSON Validation

The model self-checks its own output: balanced brackets, correct types (arrays vs strings), no trailing commas. The system prompt instructs 'Return raw JSON only — no markdown, no backticks, no explanation' to keep output clean for parsing.

Why a Single API Call?

Modern LLMs like Claude are large enough to do all 6 mental steps in one forward pass. Chaining multiple calls (one for summary, one for links, etc.) would be slower and more expensive. The structured system prompt is what makes a single call produce rich, multi-field output.

STEP 03 · FRAMEWORK FLOW — HOW THE CODE WORKS

JavaScript Framework Architecture

The agent is built as a self-contained HTML/JavaScript application. There is no backend server, no Node.js, no build step. Everything runs in the browser. Below is the complete framework flow.

1	USER ACTION	User pastes email into textarea and clicks Extract button
2	INPUT VALIDATION	JS checks textarea is not empty. If empty → shows inline error. If OK → proceeds.
3	UI STATE UPDATE	Button disabled, spinner shown, previous results cleared, status = 'Analyzing...'
4	API REQUEST BUILD	JS constructs fetch() call: model, max_tokens, system prompt, user message (the email)
5	FETCH TO ANTHROPIC	Async POST to https://api.anthropic.com/v1/messages with JSON body
6	AWAIT RESPONSE	JS awaits the Promise. Claude processes the email (2–5 seconds).
7	JSON EXTRACTION	Response data.content[0].text is extracted. Markdown fences stripped.
8	SAFE JSON PARSE	JSON.parse() in try/catch. On failure → show error box with raw output.
9	RENDER RESULTS	renderResults(data) iterates each field and builds DOM card elements.
10	UI RESET	Button re-enabled, spinner hidden, status = 'Extraction complete'.

STEP 04 · STEP-BY-STEP: BUILDING THE AGENT

How to Build It from Scratch

Follow these 7 steps in order. Each step builds on the previous one. By the end you will have a fully working AI email extraction agent.

Step 1 — Create the HTML Shell

Start with a single HTML file. No frameworks, no build tools needed.

HTML Structure

```
<!DOCTYPE html>
<html lang='en'>
<head>
  <meta charset='UTF-8'>
  <title>Email AI Agent</title>
  <style> /* your CSS here */ </style>
</head>
<body>
  <textarea id='emailInput'></textarea>
  <button onclick='runExtraction()'>Extract</button>
  <div id='results'></div>
  <script> /* your JS here */ </script>
</body>
</html>
```

Step 2 — Write the System Prompt (Agent Brain)

The system prompt is the most important part of the agent. It defines the AI's role, output format, and extraction rules. Store it as a JavaScript constant.

JavaScript — SYSTEM_PROMPT constant

```
const SYSTEM_PROMPT = `
You are an expert email analyst.
Extract all important information from the email
and return ONLY a valid JSON object:
{
  "subject": "email subject line",
  "sender": "name and email",
  "recipients": ["list"],
  "date": "date string",
  "urgency": "low | medium | high",
  "summary": "2-3 sentence overview",
  "key_headings": ["section headings"],
  "major_points": ["key ideas"],
  "action_items": [{"task", "owner", "deadline"}],
  "links": [{"url", "anchor_text", "purpose"}],
  "important_text": ["critical phrases"]
}
Return raw JSON only. No markdown. No backticks.`
```

Step 3 — Build the API Call Function

The core function sends the email to Claude and returns structured JSON. It uses the browser's native `fetch()` API — no external libraries needed.

JavaScript — Core API function

```
async function runExtraction() {
  const email = document.getElementById('emailInput').value;
  if (!email.trim()) { showError('Paste an email first'); return; }

  // Build the API request
  const response = await fetch(
    'https://api.anthropic.com/v1/messages',
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        model: 'claude-sonnet-4-20250514',
        max_tokens: 1000,
        system: SYSTEM_PROMPT,
        messages: [{
          role: 'user',
          content: `Extract info:\n\n${email}`
        }]
      })
    }
  );
  const result = await response.json();
  const raw = result.content[0].text;
  return JSON.parse(raw);
}
```

Step 4 — Add Safe JSON Parsing

LLMs sometimes wrap JSON in markdown fences. Strip these before parsing. Always wrap in try/catch to handle malformed responses gracefully.

JavaScript — Safe JSON parsing

```
function safeParse(rawText) {
  try {
    // Remove markdown backtick fences if present
    const clean = rawText
      .replace(/```json/g, '')
      .replace(/```/g, '')
      .trim();
    const data = JSON.parse(clean);
    return { success: true, data };
  } catch (err) {
    return { success: false, error: err.message };
  }
}
```

Step 5 — Manage UI State

A good agent controls the UI state throughout the extraction lifecycle: loading, success, and error states all need clear visual feedback.

State	Button	Spinner	Status Text	Results
Idle	Enabled	Hidden	—	Hidden
Loading	Disabled	Spinning	'Analyzing email...'	Cleared
Success	Enabled	Hidden	'Extraction complete'	Visible
Error	Enabled	Hidden	—	Error box shown

Step 6 — Build the Results Renderer

The renderResults() function takes the parsed JSON object and creates DOM elements for each field. Each data type gets its own visual card style.

JavaScript — Results renderer

```
function renderResults(data) {  
  const container = document.getElementById('results');  
  container.innerHTML = '';  
  
  // Render each section only if data exists  
  if (data.summary) renderSummaryCard(data.summary);  
  if (data.major_points?.length)  
    renderBulletCard('Major Points', data.major_points);  
  if (data.action_items?.length)  
    renderActionTable(data.action_items);  
  if (data.links?.length)  
    renderLinksCard(data.links);  
  if (data.important_text?.length)  
    renderHighlights(data.important_text);  
}
```

Step 7 — Style the Cards

Each result section is a styled card. Use CSS variables for theming. Keep it clean: white backgrounds, light borders, generous padding.

CSS — Card styling

```
.section-card {  
  background: var(--color-background-primary);  
  border: 0.5px solid var(--color-border-tertiary);  
  border-radius: 12px;  
  overflow: hidden;  
  margin-bottom: 12px;  
}  
.section-header {  
  display: flex; align-items: center; gap: 10px;  
  padding: 12px 20px;  
  background: var(--color-background-secondary);  
  border-bottom: 0.5px solid var(--color-border-tertiary);  
}  
.section-body { padding: 16px 20px; }
```

STEP 05 · THE SYSTEM PROMPT — AGENT'S BRAIN

Anatomy of the System Prompt

The system prompt is the single most important element of any LLM agent. Every word matters. Here is a breakdown of each component and why it exists.

Role Declaration

```
"You are an expert email analyst."
```

Sets the model persona. An expert analyst is precise, thorough, and structured — not casual or conversational. This single sentence shifts Claude's entire response style.

Task Instruction

```
"Extract all important information from the email provided"
```

Explicitly states the task. 'All' is important — without it, Claude might summarize selectively. 'Important' prevents including every trivial word.

Output Format Lock

```
"return ONLY a valid JSON object with this exact structure"
```

'ONLY' and 'exact' are critical. They prevent Claude from adding preamble ('Here is the extracted info:') or prose after the JSON. This makes machine parsing reliable.

Schema Definition

```
{ "subject", "sender", "urgency", "major_points", "action_items", "links" ... }
```

The JSON schema is copied verbatim into the prompt. Every field name, every nested structure, every data type (string vs array vs object) is specified. This eliminates field-name guessing by the model.

Hallucination Guard

```
"Only include fields that have actual content"
```

This single instruction prevents the model from inventing data. If there are no links in the email, links: [] is returned rather than a fabricated URL.

Format Enforcement

"Return raw JSON, no markdown, no backticks, no explanation"

Without this, Claude often wraps JSON in ```json ... ``` markdown fences or adds 'Here is the structured extraction:' before the JSON. This instruction strips all of that away.

STEP 06 · API CALL & JSON PARSING

The Anthropic API Request — Field by Field

Field	Value	Why It Matters
model	claude-sonnet-4-20250514	Latest Sonnet — best balance of speed, cost, and capability
max_tokens	1000	Caps output length. 1000 tokens ≈ 750 words — enough for rich JSON
system	SYSTEM_PROMPT constant	Sets the AI's role and output rules before it sees the email
messages[0].role	user	The email text is the 'user' turn in the conversation
messages[0].content	Email text	The raw email — headers, body, signature, everything
Content-Type header	application/json	Required for the API to parse the request body correctly

Response Structure from the API

The Anthropic API returns a response object. The extraction result lives at `data.content[0].text`. Here is the full response shape:

```
API Response — how to extract the text

// Full API response shape
{
  "id": "msg_abc123",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "{ \"subject\": \"Q3 Review\", ... }"
    }
  ],
  "model": "claude-sonnet-4-20250514",
  "stop_reason": "end_turn",
  "usage": { "input_tokens": 420, "output_tokens": 310 }
}

// Extract just the text:
const raw = data.content[0].text;
const parsed = JSON.parse(raw);
```

JSON Output Schema — Every Field Explained

Field	Type	Example Value	Description
subject	string	"Q3 Roadmap Review"	The email subject line

sender	string	"Sarah Chen <s@co.io>"	Full sender name + address
recipients	string[]	["team@co.io"]	All TO and CC recipients
date	string	"June 2, 2026"	Send date, as written in the email
urgency	string	"high"	low / medium / high based on language
summary	string	"2-3 sentence overview"	Concise plain-English summary
key_headings	string[]	["Action Items", "Details"]	Section titles found in body
major_points	string[]	["78% cite onboarding"]	Every significant idea
action_items	object[]	{ "task", "owner", "deadline" }	Tasks with who and when
links	object[]	{ "url", "anchor_text" }	All URLs with context
important_text	string[]	["Deadline: June 5"]	Critical verbatim phrases

STEP 07 · UI & RESULTS RENDERING

How Results Are Displayed

Once the JSON is parsed, `renderResults()` creates one card per data type. Each card only appears if the field has actual content — empty fields produce no DOM elements.

Email Metadata Card

- Displays: subject, sender, recipients, date, urgency badge
- Urgency gets color-coded pill: green=low, amber=medium, red=high
- Uses CSS Grid with auto-fit columns for responsive layout

Summary Card

- 2-3 sentence plain-English overview of the entire email
- Largest text on the page — most important fast-read element
- Always rendered first after metadata

Major Points Card

- Bullet list of every significant idea in the email
- Each bullet is a complete sentence extracted from the body
- Custom dot bullets styled with `::before` CSS pseudo-elements

Action Items Card

- Rendered as a three-column table: Task | Owner | Deadline
- Owner and Deadline shown as small pill badges with icons
- Empty cells display an em dash — never left blank

Important Highlights Card

- Critical phrases shown in left-border accent boxes
- Visual weight signals urgency without color alone
- Preserves near-verbatim language from the original email

Links Card

- Each link shown in its own mini-card: anchor text, URL, purpose
- URLs in monospace font for readability
- Purpose field explains why the link exists in context

STEP 08 · ERROR HANDLING & EDGE CASES

Handling What Can Go Wrong

Empty Input	Scenario: User clicks Extract with no email pasted	Fix: Check textarea value before API call. Show inline error message.
API Auth Error	Scenario: 401 response — API key missing or wrong	Fix: Check response.ok. If false, read response.json() for error.message and display it.
Malformed JSON	Scenario: LLM returns JSON with markdown fences or extra text	Fix: Strip ```json and ``` with regex before JSON.parse(). Wrap in try/catch.
Missing Fields	Scenario: Email has no links → links field absent from JSON	Fix: Always check field existence: if (data.links && data.links.length) before rendering.
Very Long Email	Scenario: Email exceeds model context window	Fix: Truncate email to ~3000 words before sending, or chunk into multiple passes.
HTML Email	Scenario: Email body contains raw HTML tags	Fix: Strip HTML with: email.replace(/<[>]*>/g, '').replace(/\s+/g, ' ').trim()
Network Error	Scenario: fetch() throws — offline or CORS issue	Fix: Wrap entire fetch in try/catch. Show user-friendly error, not the raw exception.

Golden Rule for Error Handling

Never let the UI freeze or show a blank screen. Every error path must:

1. Re-enable the Extract button
2. Hide the spinner
3. Show a clear message

Users should always know what happened and be able to try again.

STEP 09 · COMPLETE CODE REFERENCE

Full Agent Code — Annotated

Below is the complete, production-ready JavaScript for the agent. Every function is explained with inline comments.

Complete `runExtraction()` with Full Error Handling

JavaScript — Complete production function

```
async function runExtraction() {
  const email = document.getElementById('emailInput').value.trim();

  // Guard: empty input
  if (!email) { showError('Please paste an email first.');
```

```
  return; }

  // Update UI: loading state
  hideError();
  setButtonState(false); // disable button
  showSpinner(true);
  clearResults();
  setStatus('Analyzing email...');

  try {
    const response = await fetch(
      'https://api.anthropic.com/v1/messages', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        model: 'claude-sonnet-4-20250514',
        max_tokens: 1000,
        system: SYSTEM_PROMPT,
        messages: [{ role: 'user',
          content: `Extract info from:\n\n${email}` }]
      })
    );

    // Check HTTP-level errors
    if (!response.ok) {
      const err = await response.json();
      throw new Error(err.error?.message || 'API error');
    }

    // Extract and parse JSON
    const result = await response.json();
    const raw = result.content.map(b => b.text||'').join('');
    const clean = raw.replace(/```json|```/g, '').trim();
    const data = JSON.parse(clean);

    setStatus('Extraction complete');
    renderResults(data); // render all cards

  } catch (err) {
    showError('Failed: ' + err.message);
    setStatus('');
  } finally {
    setButtonState(true); // always re-enable button
    showSpinner(false);
  }
}
```

HTML Escaping — Prevent XSS

Any text inserted into the DOM from an external source (the email) must be escaped to prevent cross-site scripting. Always sanitize before using innerHTML.

JavaScript — XSS prevention

```
function esc(str) {  
  if (!str) return '';  
  return String(str)  
    .replace(/&/g, '&amp;')  
    .replace(/</g, '&lt;')  
    .replace(/>/g, '&gt;')  
    .replace(/"/g, '&quot;');  
}  
  
// Usage: always escape email-derived strings  
element.innerHTML = `<span>${esc(data.sender)}</span>`;
```

STEP 10 · EXTENDING THE AGENT

How to Add More Capabilities

The agent is designed for extension. Because intelligence lives in the prompt, most additions require only prompt edits — no architectural changes.

Add Sentiment Analysis

```
Add to schema: "sentiment": { "tone": "positive|neutral|negative", "confidence": 0.0-1.0 }
```

Add the field to the JSON schema in your SYSTEM_PROMPT. Claude will auto-populate it.

Support Multiple Languages

```
Add to prompt: "If the email is not in English, translate all extracted text to English"
```

Claude handles translation inline — no separate translation API call needed.

Priority Scoring

```
Add to schema: "priority_score": 1-10 with reasoning in "priority_reason"
```

Ask Claude to rate the email 1-10 based on urgency, deadlines, and stakeholder count.

Export to JSON File

```
const blob = new Blob([JSON.stringify(data,null,2)], {type:'application/json'})
```

Add a Download button that creates a Blob from the parsed data and triggers a file save.

Email Thread Support

```
Add to prompt: "Focus on the most recent message in the thread. Note if it is a reply."
```

Threaded emails contain multiple messages — instruct Claude which to prioritize.

Slack/Webhook Integration

```
After extraction, POST the JSON to a Slack webhook or Zapier endpoint
```

Add a second fetch() call after renderResults() to forward the structured data downstream.

Summary: The Email AI Extraction Agent combines a carefully engineered system prompt, a single Anthropic API call, safe JSON parsing, and a dynamic results renderer to transform any raw email into a fully structured, categorised intelligence report — in under 5 seconds, with zero backend infrastructure required.